Assignment 3

Part 1: Reading Assignment on Transformer Jupernotebook

1. Transformer Architecture

Two main components:

- Encoder Stack (left): Processes input
- Decoder Stack (right): Generates output step-by-step

2. Inside the Encoder Block

Each Encoder Block contains:

- Self-Attention Layer
- Feed Forward Neural Network (applied position-wise)

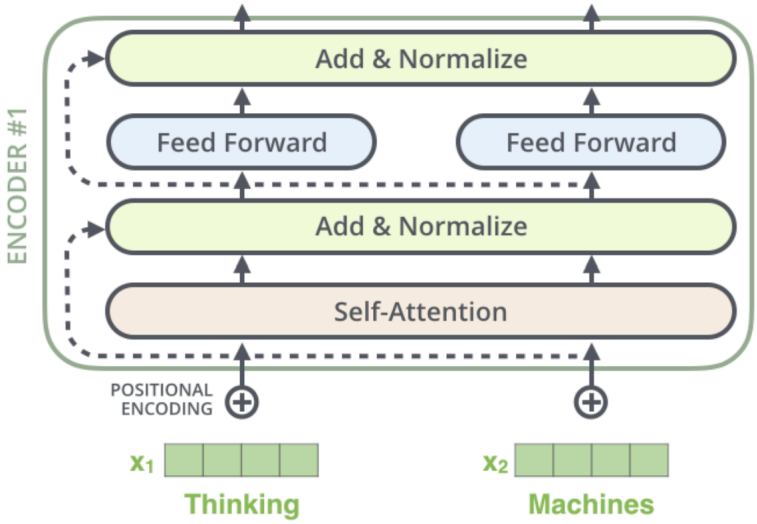
Each word can see all other words in the input sentence using self-attention.

3. Add & Normalize Layer (Residual Connection)

After each sub-layer of an encoder, we add to the output the input and then apply layer normalization to it

Layer Norm = $scale \ * \ \frac{x - mean(x)}{std(x) + \epsilon} + \ shift$

- Scale and shift are learnable parameters
- Helps stabilize and speed up training



4. Self-Attention Mechanism

For each word

- It creates Query vector (qi), key vector (ki) and Value vector (vi)

Score Calculation

$$Score(qi, \ kj) \ = \ qi \ . \ ki$$

Scaled attention score:

$$Scaled \ score \ = \ \frac{qi \ . \ ki}{\sqrt{dk}}$$

Where dk is dimension of key vectors

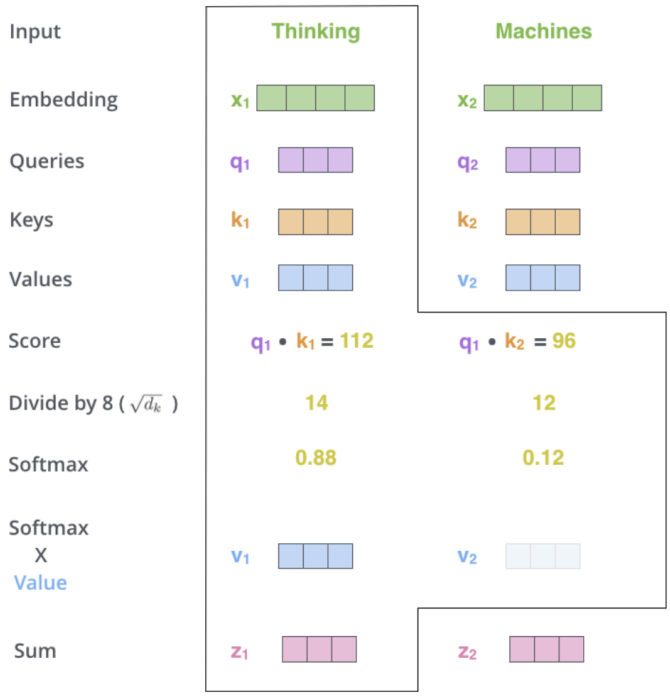
Softmax over scores:

$$Attention\ weights\ = Softmax(\frac{qi\ .\ ki}{\sqrt{dk}})$$

Weighted Sum:

$$zi\ = \sum_{j} Attention\ Weights_{ij}\ \times\ vj$$

Each word becomes a weighted sum of all other words' value vectors based on attention.



5. Multi-Headed Attention

- Instead of one attention, use multiple heads.
- Each head projects into different subspaces.
- Outputs are concatenated and passed through a final linear layer.

Captures multiple types of relationships between words simultaneously.

6. Inside the Decoder Block

Each Decoder Block contains:

- Masked Self-Attention (prevent looking ahead)
- Encoder-Decoder Attention (attend to encoder outputs)
- Feed Forward Layer
- Add & Normalize Layers

6.1. Masked Self-Attention (in Decoder)

- Mask future words during training so the decoder predicts words sequentially.
- Subsequent mask matrix is lower triangular.
- Decoder only attends to already seen (past) words.

7. Generator (Final Linear + Softmax)

After decoding:

- Pass through a **Linear layer** → output logits over vocabulary
- Apply **Softmax** to get probabilities

Formula:

PredictedProbabilities = Softmax(Linear(x))

8. Feed Forward Network (FFN)

- Each position separately applies:
- Linear layer (expand)
- ReLU activation
- Linear layer (project back)

Formula:

$$FFN(x) = Linear_2(ReLU(Linear_1(x)))$$

Input and output size = 512
Hidden layer size = 2048

9. Positional Encoding

Since transformers don't inherently understand word order, we add positional information using sine and cosine waves.

Formulas:

Even indices (dimension 2i):

$$PE_{(pos,2i)} = Sin\left(\frac{pos}{1000^{\frac{2i}{d_{model}}}}\right)$$

Odd indices (dimension 2i+1):

$$PE_{(pos,2i)} = Cos\left(\frac{pos}{1000^{\frac{2i}{d_{model}}}}\right)$$

where:

- pos = position
- d_{model} = embedding size (512)
- Allows model to generalize to different sequence lengths and understand order.

10. Final Transformer Model Assembly

Main Components:

- Encoder Stack: 6 identical encoder layers
- Decoder Stack: 6 identical decoder layers
- Embeddings + Positional Encodings
- Generator (Linear + Softmax)

Number of Attention Heads: 8
Dimension of Model (d_model): 512
Feedforward Hidden Size: 2048

Part 2

Section 2.1: Pytorch LSTM

LSTM Hyperparameters

Hyperparameter	Value
Embedding Size	300 (GloVe 6B 300d)
Vocab Size	14921
Hidden Dimension	256
Number of Layers	1
Dropout Rate	0.3
Pretrained Embeddings	yes
Batch Size	64
Learning Rate	0.004
Number of epochs	30
Weight Decay	1e-5
Optimizer	Adam

Performance Metrics

Accuracy : 43.98%
Test loss : 1.451
Test Macro F1-Score : 0.3847
Test Weighted F1-Score : 0.4105

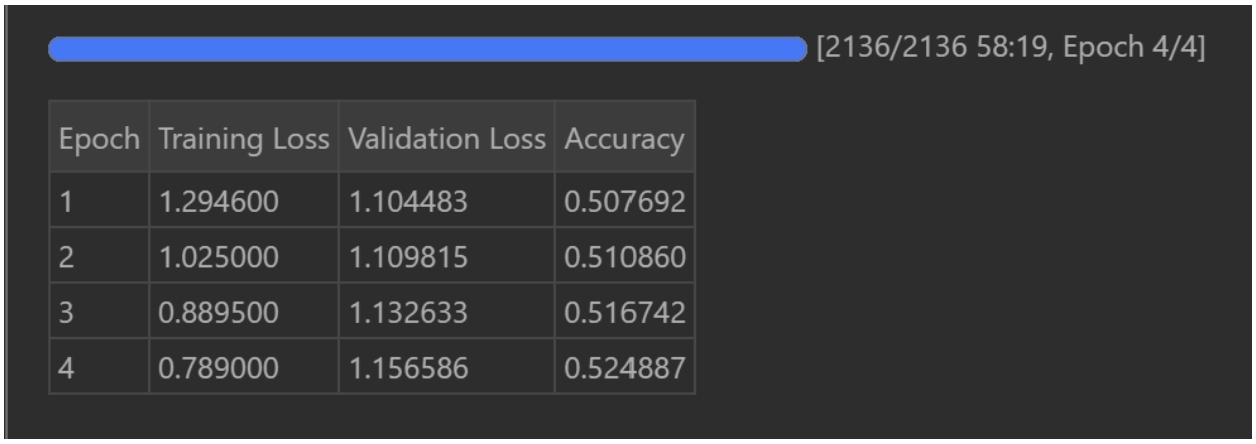
```
Test Loss: 1.4511013780321393

------Test------
          class precision  recall f1-score support
0             0    0.4359  0.2437   0.3126     279
1             1    0.4621  0.6445   0.5383     633
2             2    0.3583  0.1105   0.1690     389
3             3    0.3801  0.6431   0.4778     510
4             4    0.6649  0.3133   0.4259     399
5      accuracy                      0.4398    2210
6     macro avg    0.4603  0.3910   0.3847    2210
7  weighted avg    0.4582  0.4398   0.4105    2210
```

We fine-tuned a 1-layer LSTM model using pretrained GloVe 300d embeddings on the SST-5 dataset. The model achieved a test accuracy of 43.98% and a macro F1-score of 0.3847 after 30 epochs. The best performance was observed for negative and positive classes, while the neutral class remained the most challenging.

Section 2.2: BERT-Related Training Hyperparameters

Hyperparameter	Value
Model used	bert-base-cased
Tokenizer used	bert-base-cased
Pre trained model parameters	True
Number of Labels	5
Pretrained Embeddings	yes
Batch Size	16
Learning Rate	1e-5
Number of epochs	4
Weight Decay	0.01

Performance Metrics:



Epoch	Training Loss	Validation Loss	Accuracy
1	1.294600	1.104483	0.507692
2	1.025000	1.109815	0.510860
3	0.889500	1.132633	0.516742
4	0.789000	1.156586	0.524887

Part 3:

Section 3.3 LLM-based Exam Agent For NLP

Step 1: Testing with Atomic Tools Only (extract_text, count_occurrences, divide, multiply)

- I tested simple arithmetic tasks like "Multiply 6790 by 5" and "Divide 15 by 5".
- GPT-3.5-turbo successfully called the tools multiply and divide correctly.
- The model did not try to compute internally; instead, it invoked the appropriate atomic tools as expected.
- I also tested an N-gram task by asking it to "According to the screenshot of the exam, solve the ngram questions steps by steps"
- It correctly called the extract_text tool to extract the text content from the image.
- However, since bigram-specific tools were not yet available, GPT-3.5-turbo could not fully compute bigram probabilities at this stage — it was limited by the available tools.
- Overall, GPT-3.5-turbo showed a good ability to use basic tools when available.

Step 2: After Implementing Bigram-Related Tools

- I implemented the missing functions: bigram_condition_prob_in_corpus, bigram_prob, and bigram_prob_smooth from assignment 1
- I updated the tool list to expose these new functions to the agent.
- I again asked the agent to solve the N-gram questions from the exam screenshot.
- GPT-3.5-turbo correctly called the advanced bigram tools:
    - It used bigram_condition_prob_in_corpus to compute $P(\text{some}|\text{and})$
    - It used bigram_prob_smooth to compute probabilities for the bigrams "some are" and "are good" with add-1 smoothing.
    - It used multiply to combine the individual bigram probabilities into the full sentence probability.
- The results it computed made sense and were consistent with the expected calculations.
- Notable observation: GPT-3.5-turbo needed multiple tool calls (in sequence) to finish the task, showing its ability to chain the reasoning properly.